MARA - Master Build & Career Guide

Final-Year B.Tech CSE Team of 5 Consolidated from all planning sessions (Sep 2026)

This document is the single source of truth for what to build, in what order, and why. It supersedes the phase ordering in the original build guide where the two differ. The original guide (MARA_Complete_Build_Guide.pdf) remains the reference for per-phase code and setup details.

---

1. Where we are right now (status)

| --- | --- | --- |
Scaffolding, config, venv (Python 3.12)	1	[OK] Built
arXiv ingestion + embeddings + Qdrant indexing	2-3	[OK] Built
Planner agent (defensive JSON parsing)	4	[OK] Built
Retriever agent (fail-soft, idempotent indexing)	3	[OK] Built
Critic agent (fail-closed verdicts)	5	[OK] Built
Offline test suite	-	[OK] 52 tests passing
Writer agent	6	[ ] Next
LangGraph orchestration	7	[ ]
FastAPI backend	8	[ ]
Streamlit frontend	9	[ ]
Deployment	10	[ ]
Evaluation framework	11	[ ]
Report & viva	12	[ ]
Git commits / GitHub	-	[!] Nothing committed yet - identity unset

---

2. Top priority - must do, in this order

P0 - Foundation (week 1; unblocks the whole team)

1. Git + GitHub + CI. Set git identity, push to GitHub, protect main, everyone works in feature branches with PRs. Add a GitHub Actions workflow running the offline tests on every PR (free, ~15 min). Nothing else matters until this is done.
2. Commit all existing work (everything is already staged).
3. Writer agent (Phase 6) - same conventions as the other three agents:

lazy client, defensive parsing, retry, offline tests, standalone demo.

4. LangGraph orchestration (Phase 7) with one critical design fix: the original guide's critic node checks each chunk against itself (check_claim(chunk["text"], [chunk["text"]])) - always SUPPORTED, tautological, useless. Instead: generate candidate claims from the evidence first, then verify each claim against all retrieved chunks (cross-verification). This is a headline viva point: "the naive design self-verifies; ours cross-verifies."

P1 - Make it run end-to-end (weeks 2-3)

5. Mock mode (MOCK_LLM=1) - deterministic stubbed agent responses so all five members develop and demo with zero API cost and zero keys.

6. Per-run trace logging - log every agent's inputs/outputs/timing to a JSON file per query. This automatically generates the Phase 11 evaluation data (latency per agent, factual-consistency rate, source counts).

7. FastAPI backend (Phase 8) - /query endpoint plus a /trace view of the pipeline steps.

8. Streamlit frontend (Phase 9) with streaming progress - show "Planner v -> Retriever v -> Critic 4/12 -> Writer v" live; a 30-60 s silent spinner looks broken during the viva demo.

9. Confidence badge - "11/12 claims verified (92%)" computed from Critic verdicts; visible in the UI; quantifies trust.

10. Evaluation framework (Phase 11) + naive single-prompt baseline - RAGAS-style metrics (faithfulness, relevancy); the baseline proves the multi-agent design adds value rather than complexity.

P2 - The standout trio (what makes it resume/interview-grade)

11. Fine-tune your own embedding model - THE headline ML feature. The project today is 100% orchestration and 0% machine learning; this is the answer to "where is the ML in your ML project?" Steps:

* Label ~300 query -> relevant-chunk pairs (split across the team; this is spreadsheet work, and the same labels double as Phase 11 evaluation data).
* Fine-tune bge-small-en (free, 33 M params, trains on a laptop CPU) with contrastive loss via sentence-transformers (MultipleNegativesRankingLoss).
* Measure recall@5 / MRR before vs after (e.g., 0.62 -> 0.84) and deploy the fine-tuned model in the retriever.
* Resume line: *"Trained a custom retrieval model on 300+ labeled query-passage pairs, improving recall@5 by X% and end-to-end

faithfulness by Y%."*

12. Ablation study: Critic ON vs OFF - quantify the faithfulness drop when the Critic is disabled. Real research methodology; proves the architecture with numbers instead of claims.

13. Citation auto-verification - after the Writer finishes, extract every [Source: ...] citation and automatically verify the cited source supports that sentence. Automates the one evaluation metric the original guide says to check manually.

P3 - Deployment + demo insurance

14. Deploy (Phase 10): Render backend, Streamlit Community Cloud frontend, Qdrant Cloud free tier.

15. Ollama offline mode (LLM_PROVIDER=ollama) - run the entire demo with no internet and no API keys. Venue Wi-Fi dying during the viva is a real risk; be the only team still demoing.

---

3. Further features (build only if time remains)

Strong differentiators

* Self-revision loop - Writer drafts -> Critic re-checks the final answer -> if faithfulness is below a threshold, LangGraph loops back with feedback. Turns the linear graph into a true agent cycle; the best possible answer to "why LangGraph?"

* Contradiction detector - when sources disagree, surface it ("Source A: 12% improvement; Source B: 4%") instead of averaging. Rare in student projects, academically honest, memorable demo moment.

* PDF full-text ingestion - index paper bodies (pypdf, section-aware chunking), not just 200-word abstracts; answers grounded in methods/results sections separate you from every abstract-only RAG project.

* Evidence table UI - claim -> verdict -> source -> exact supporting quote, clickable/highlighted. Makes traceability visible; the thing the panel will remember.

* Hybrid search + reranking - BM25 + dense vectors, rerank top-20 with a local cross-encoder (bge-reranker-base, free, CPU-friendly); measurable retrieval-precision gain.

* Semantic caching - embed incoming queries; if cosine similarity to a past query exceeds a threshold, return the cached answer; report latency/cost savings (threshold-vs-staleness trade-off is a tunable you can

graph).

Polish / optional

* Conversational follow-ups (carries context; the base paper is conversational while the guide's MARA is one-shot - this closes that gap).
* Citation-graph expansion via Semantic Scholar when retrieval comes back thin.
* BibTeX export of every answer's sources (~20 lines).
* SQLite query history (past answers browsable; doubles as evaluation data).
* Field-trend analysis (the corpus stores dates/categories - "how did RAG evaluation evolve 2020->2026?").
* Token/cost dashboard (measured, not estimated, for the report's cost table).
* Reproducibility manifest per run (model versions, corpus hash, settings).

Skip entirely (overkill for a B.Tech project)

Multi-user auth React rewrite WebSocket streaming at scale * fine-tuning an LLM * Kubernetes. They cost weeks and add zero evaluation value.

---

4. Team roles (team lead = core pipeline)

| --- | --- | --- |
You (lead)	Core AI	Writer, LangGraph, FastAPI, fine-tuning
A	Evaluation lead	Benchmarks, labeling, ablations, results + charts
B	Frontend lead	Streamlit app, streaming progress, demo UX
C	Data lead	Semantic Scholar ingestion, corpus curation, chunk e
D	Report/docs lead	Report chapters, diagrams, slides, viva Q&A bank
E	QA lead	Tests, failure analysis, Ollama offline mode, live-dem

Rules that make this real rather than cosmetic:

* Everyone produces measurable artifacts (labels, tests, eval runs) for the modules they observe.
* Report chapters map 1:1 to the roles - evaluators notice when all five members contributed concrete deliverables.
* Use GitHub Issues as the kanban: one issue per phase checkpoint from the original guide's timeline.

---

5. Resume bullets to aim for (end state)

1. "Built MARA, a 4-agent RAG research assistant (Planner / Retriever / Critic / Writer) with cross-verification, raising answer faithfulness to X% versus a single-prompt baseline."
2. "Trained a custom retrieval embedding model on 300+ labeled query-passage pairs (contrastive learning), improving recall@5 from X to Y."
3. "Ablations (Critic on/off, baseline comparison) with automated per-run tracing; cut median latency Z% via semantic caching."
4. "Engineered for reliability: 52+ offline tests in CI, mock-LLM mode, and an offline (Ollama) demo mode."

Fill in X/Y/Z with the real measured numbers at the end.

---

6. Viva one-liners

* "Why not just one LLM call?" -> Hallucination risk and zero traceability; our metrics prove the gap versus the baseline.
* "Why LangGraph?" -> Explicit state graph with conditional edges enables our revision/adequacy loops; fully autonomous agent loops are hard to debug and reason about.
* "What if retrieval finds nothing relevant?" -> The retriever fails soft and the pipeline degrades to the "insufficient verified evidence" fallback.
* "Where is the ML?" -> We fine-tuned and deployed our own embedding model on our own labeled data, with before/after recall measurements.

---

7. Suggested order (one page)

| --- | --- |
1	GitHub + CI + first commit
2-3	Writer + LangGraph (cross-verification) + mock mode + tracing
4-5	FastAPI + Streamlit (streaming) + confidence badge
6-7	Evaluation harness + naive baseline; labeling starts in parallel (Member A)
8-9	Fine-tune embeddings; deploy
10-11	Ablation + citation auto-verification + evidence table
12+	Differentiators from §3, report, dry-run demos with Ollama fallback

---

Keep notes as you build - they become the "Challenges Faced" section of the final report.